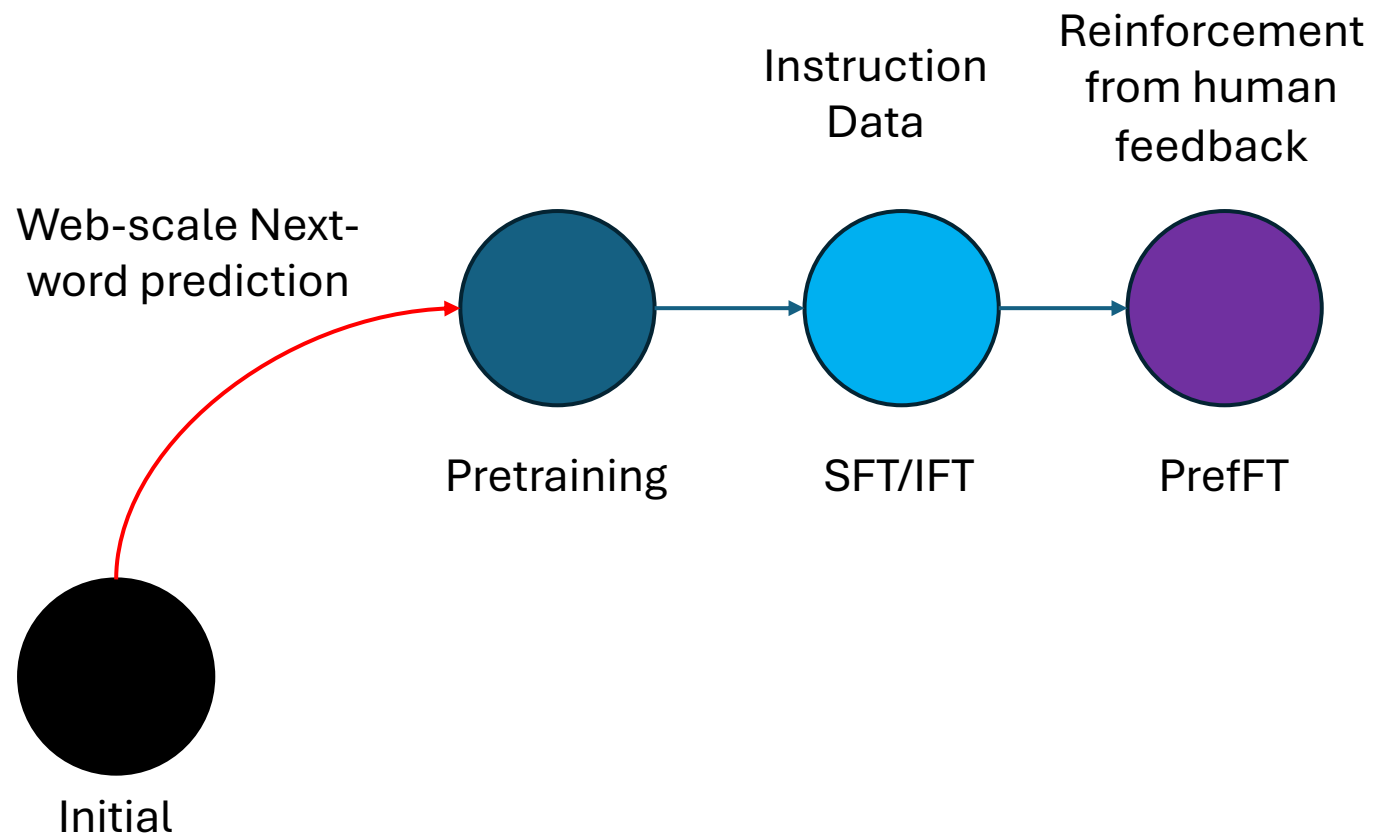


The path to ChatGPT



Scaling laws: predicting loss from compute, data and parameters

- Empirical **power laws**: test loss falls as a straight line in log-log with data, parameters, and compute (Kaplan et al., 2020)
- Chinchilla (Hoffmann et al., 2022): for a fixed compute budget there is an **optimal model size and token count** (≈ 20 tokens/parameter)
- Why we care: decide *bigger model vs. more data*, pick hyperparameters on small models and transfer up, and forecast what a training run will give
- The next slides follow Stanford CS336 (Lecture 9 and 11, Spring 2025)

Supervised Fine-Tuning

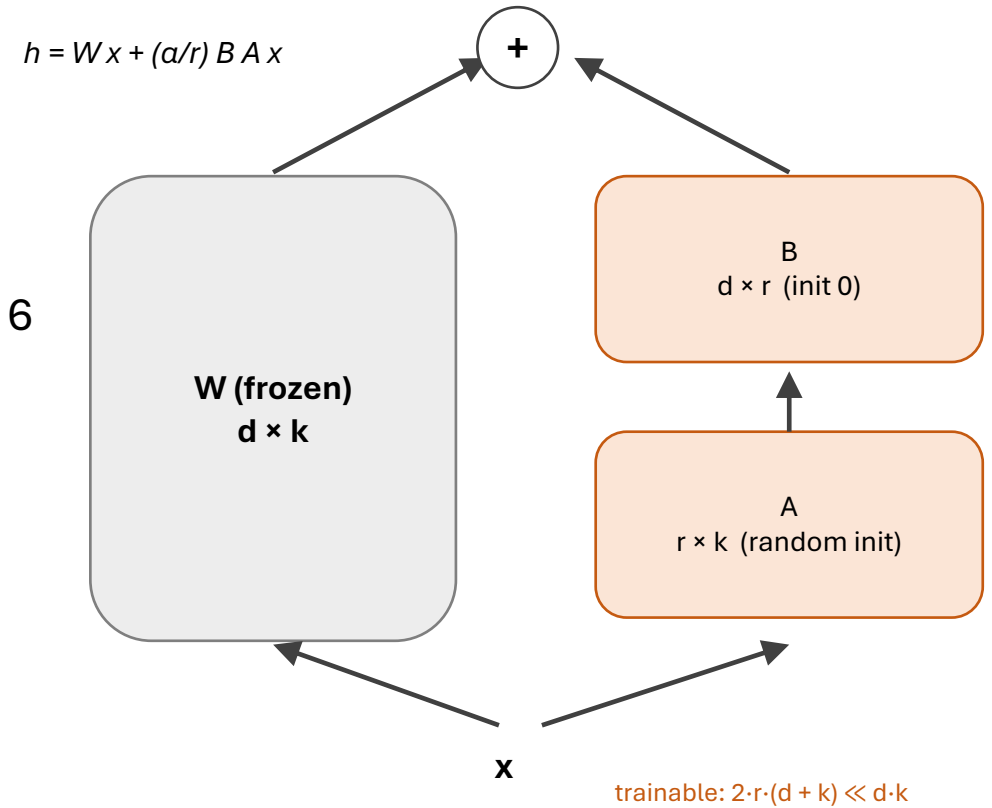
- Collect pairs of input/outputs with desired behavior (aka SFT data)
- Train using next word prediction objective given the input
- Special case. SFT on **instruction-following data**.
- **Questions:**
 1. What does that data look like?
 2. How do we best make use of that data?
 3. Do we need scale for this?

SFT is behavior cloning

- Supervised fine-tuning maximizes $\log \pi(\text{token} \mid \text{prompt, previous tokens})$ on expert demonstrations — exactly behavior cloning on text
- The same failure mode: at test time the model conditions on **its own** previous tokens, never seen in training (exposure bias) $\rightarrow$ errors compound over long generations (the $O(\epsilon T^2)$ argument)
- The same fixes: better and more diverse demonstrations; data that shows recovery from mistakes; label the model's own outputs (DAgger) — RLHF's preference data on model samples is a form of this; or optimize the outcome directly with RL
- Lecture 11's 'Issues with SFT' slide is the LLM version of this slide

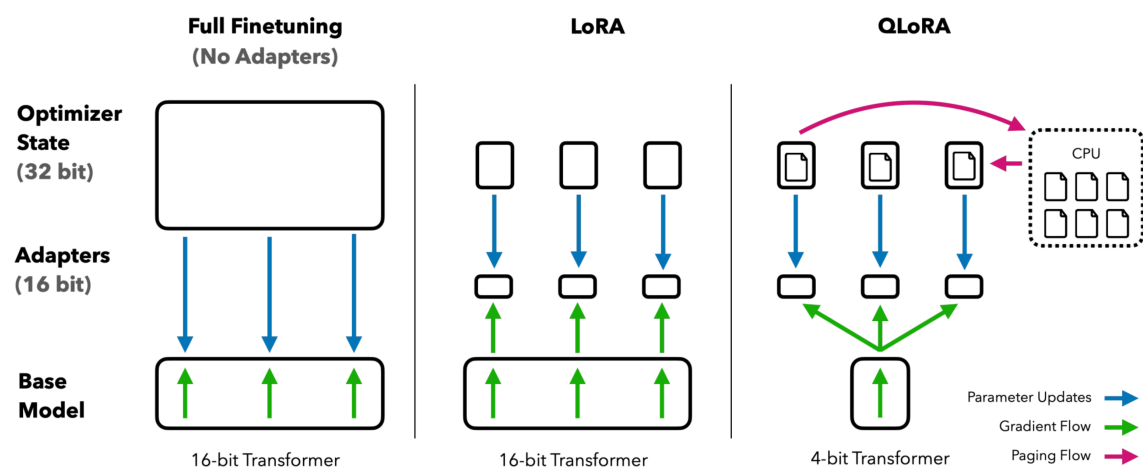
LoRA in one picture

- $h = Wx + (\alpha/r) \cdot B A x$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$; A random, $B = 0 \rightarrow$ training starts at the base model
- Example: a 4096×4096 layer has 16.8M weights; a rank-16 adapter has $2 \cdot 4096 \cdot 16 = 131k$ (0.8%)
- Paper headline: on GPT-3 175B, $10,000\times$ fewer trainable parameters and $3\times$ less GPU memory than full fine-tuning, with on-par quality
- Motivation: fine-tuning has a low intrinsic dimension (Aghajanyan et al., 2020)



QLoRA

LoRA, with quantized backbones and paged optimizer states



Model	Size	Elo
GPT-4	-	1348 $\pm$ 1
Guanaco 65B	41 GB	1022 $\pm$ 1
Guanaco 33B	21 GB	992 $\pm$ 1
Vicuna 13B	26 GB	974 $\pm$ 1
ChatGPT	-	966 $\pm$ 1
Guanaco 13B	10 GB	916 $\pm$ 1
Bard	-	902 $\pm$ 1
Guanaco 7B	6 GB	879 $\pm$ 1

- Following LoRA's design principle, but quantize the backbone models.
- NormalFloat (NF4), a new data type for LLM weight quantization
- Double quantization (scaling factors also quantized) to further reduce
- Paged Optimizers with CPU offloading

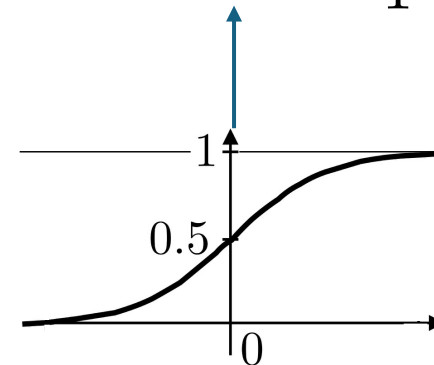
• **Enable fine-tuning LLMs on middle / entry-level GPUs.**

Step 1: Reward modeling

- **Bradley-Terry formulation.** Probability that y_i better than y_j is defined as:

$$p(y_i > y_j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j)$$

- Where r_i, r_j rewards of y_i, y_j respectively, and $\sigma(x) = \frac{1}{1 + e^{-x}}$



Proximal Policy Optimization (PPO)

- **1) Proximal:** Don't deviate too much from base model

- Avoid "reward hacking"
- Training instability

$$\text{KL} \left(\pi_{\theta}(\hat{y}|x) || \pi_{\text{ref}}(\hat{y}|x) \right)$$

- **2) Policy Optimization**

$$r(x, \hat{y})$$

- Overall,

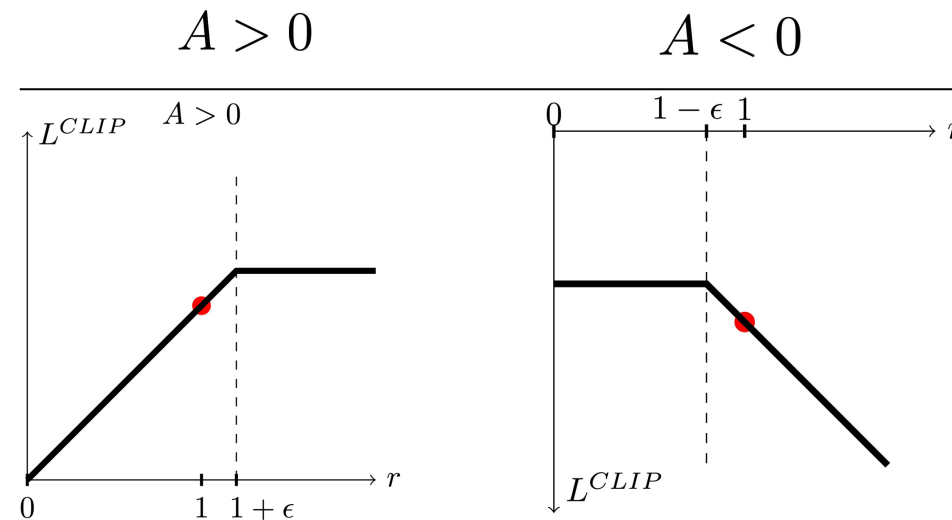
$$\min \mathcal{L}(\theta) = - \left[r(x, \hat{y}) - \lambda \text{KL} \left(\pi_{\theta}(\hat{y}|x) || \pi_{\text{ref}}(\hat{y}|x) \right) \right]$$

Variant1: PPO-Clip

- Optimize the following

$$\max \mathcal{J}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

- Where, $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$
- Intuitively, r_t measures the policy's change in step t



Direct Preference Optimization (DPO)

- DPO loss function

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- No need to train a separate reward model
- Operates directly on preference data
- Like the Bradley-Terry formulation with special kind of reward

$$r_{\theta}(x, y) = \beta \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$$

Baselines

a convenient identity

$$p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau)$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(\tau) [r(\tau) - b]$$

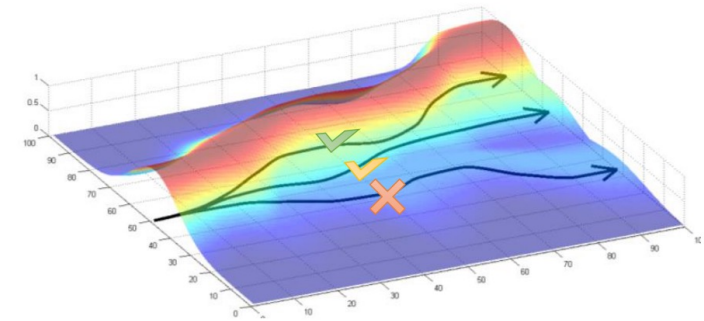
$$b = \frac{1}{N} \sum_{i=1}^N r(\tau)$$

but... are we allowed to do that??

$$E[\nabla_{\theta} \log p_{\theta}(\tau) b] = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b d\tau = \int \nabla_{\theta} p_{\theta}(\tau) b d\tau = b \nabla_{\theta} \int p_{\theta}(\tau) d\tau = b \nabla_{\theta} 1 = 0$$

subtracting a baseline is unbiased in expectation!

average reward is not the best baseline, but it's pretty good!

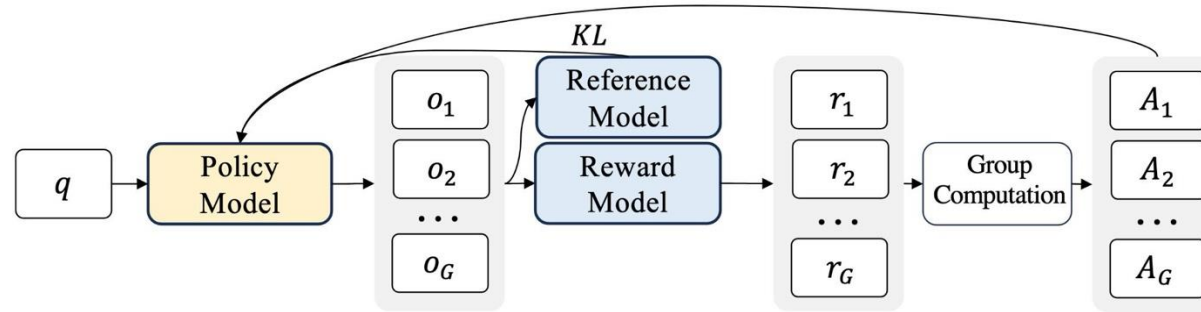


From REINFORCE to PPO to GRPO

- **REINFORCE**: $\nabla J = E[\sum_t \nabla \log \pi(a_t|s_t) \cdot R]$ — unbiased, high variance; reward-to-go and a baseline $b(s)$ fix the variance
- **Actor-critic**: learn $V(s)$ as the baseline $\rightarrow$ advantage $A = R - V$; GAE trades bias for variance
- **PPO** (Lecture 11): the importance ratio $\pi_\theta / \pi_{\text{old}}$ with **clipping** (a cheap trust region), a value network (critic), and a KL penalty to the reference model
- **GRPO** (Lecture 12): sample G responses per prompt; advantage $= (r - \text{mean}_G) / \text{std}_G$ — the **group replaces the critic**; ratio, clipping and KL stay; with a verifier as reward this is RLVR (DeepSeek-R1)

How GRPO works

GRPO

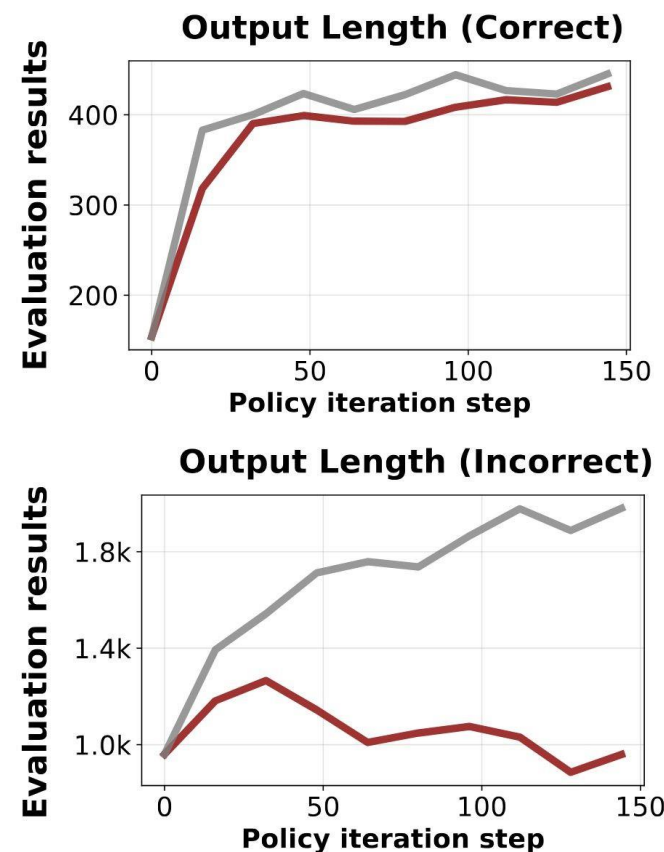


Mitigating the length problem

- Equalize token-level contributions

- DAPO
$$\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|}$$

- Dr. GRPO
$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|}$$



R1-Zero: RL with verifiable rewards

